MITRC
MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Chakravarty Sharma

Subject & Subject Code: Introduction to Python Programming & 4CS4-05

Semester: IV **Session:** 2022-23

Unit No.: 3 **Date of Submission:** 20/03/2023

Lecture No.	Topics Covered	Page No.	Total Page
17	List: Create,Accesss, slicing,. Negative indices, List Methods, and comprehensions.	2- 8	7
18	List Methods with examples	9-16	8
19	Tuples: Create, Indexing ans Slicing, Operation on tuples.	17-22	6
20	Tuples: Create, Indexing ans Slicing, Operation on tuples with examples.	23-29	7
21	Dictionary: Create,add, and replace values, operation on dictionaries	30-34	5
22	Dictionary: Methods with examples	35-40	6
23	Sets: Create and operations on set.	41-44	4
24	Set Methods with examples	45-51	7

References:

1. Programming Python ny Mark Lutz, O`Reilly
2. Intrduction to Programming using Python by Y.Daniel Liang, Pearson, 2012

Faculty **HOD** **Dean Academics**

Lecture No. 17

List: Create,Accesss, slicing,. Negative indices, List Methods, and comprehensions.

Python Lists

Lists are used to store multiple items in a single variable.

Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are Tuple, Set, and Dictionary, all with different qualities and usage.

Lists are created using square brackets [ ]:

Example

```
thislist = ["apple", "banana", "cherry"] # Create a List
print(thislist)
```

List Items

List items are ordered, changeable, and allow **duplicate values**.

List items are indexed, the first item has index [0], the second item has index [1] etc.

Ordered

When we say that lists are ordered, it means that the items have a defined order, and that order will not change.

If you add new items to a list, the new items will be placed at the **end of the list**.

Note: There are some **list methods** that will change the order, but in general: the order of the items will not change

Changeable

The list is changeable, meaning that we can change, add, and remove items in a list after it has been created

Allow Duplicates

Since lists are indexed, lists can have items with the same value:

Example

Lists allow duplicate values:

```
thislist = ["apple", "banana", "cherry", "apple", "cherry"]
print(thislist)
```

List Length

To determine how many items a list has, use **the len()** function:

Example

Print the number of items in the list:

```
thislist = ["apple", "banana", "cherry"]
print(len(thislist))
```

List Items - Data Types

List items can be of any data type:

Example

String, int and boolean data types:

```
list1 = ["apple", "banana", "cherry"]
list2 = [1, 5, 7, 9, 3]
list3 = [True, False, False]
```

A list can contain different data types:

Example

A list with strings, integers and boolean values:

```
list1 = ["abc", 34, True, 40, "male"]
```

type()

From Python's perspective, lists are defined as objects with the data type 'list':

<class 'list'>

Example

What is the data type of a list?

```
mylist = ["apple", "banana", "cherry"]
print(type(mylist))
```

The list() Constructor

It is also possible to use the list() constructor when creating a new list.

Example

Using the list() constructor to make a List:

```
thislist = list(("apple", "banana", "cherry")) # note the double round-brackets
print(thislist)
```

Python Collections (Arrays)

There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.
- **Dictionary** is a collection which is ordered** and changeable. No duplicate members.

Access Items

List items are indexed and you can access them by referring to the index number:

Example

Print the second item of the list:

```
thislist = ["apple", "banana", "cherry"]
print(thislist[1])
```

Note: The first item has index 0.

Negative Indexing

Negative indexing means start from the end

-1 refers to the last item, -2 refers to the second last item etc.

Example

Print the last item of the list:

```
thislist = ["apple", "banana", "cherry"]
print(thislist[-1])
```

Range of Indexes

You can specify a range of indexes by specifying where to start and where to end the range.

When specifying a range, the return value will be a new list with the specified items.

Example

Return the third, fourth, and fifth item:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[2:5])
```

Note: The search will start at index 2 (included) and end at index 5 (not included).

Remember that the first item has index 0.

By leaving out the start value, the range will start at the first item:

Example

This example returns the items from the beginning to, but NOT including, "kiwi":

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[:4])
```

By leaving out the end value, the range will go on to the end of the list:

Example

This example returns the items from "cherry" to the end:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[2:])
```

Range of Negative Indexes

Specify negative indexes if you want to start the search from the end of the list:

Example

This example returns the items from "orange" (-4) to, but NOT including "mango" (-1):

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[-4:-1])
```

Check if Item Exists

To determine if a specified item is present in a list use the in keyword:

Example

Check if "apple" is present in the list:

```
thislist = ["apple", "banana", "cherry"]

if "apple" in thislist:
  print("Yes, 'apple' is in the fruits list")
```

Change Item Value

To change the value of a specific item, refer to the index number:

Example

Change the second item:

```
thislist = ["apple", "banana", "cherry"]
thislist[1] = "blackcurrant"
print(thislist)
```

Output

['apple', 'blackcurrant', 'cherry']

Change a Range of Item Values

To change the value of items within a specific range, define a list with the new values, and refer to the range of index numbers where you want to insert the new values:

Example

Change the values "banana" and "cherry" with the values "blackcurrant" and "watermelon":

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
thislist[1:3] = ["blackcurrant", "watermelon"]
print(thislist)
```

If you insert *more* items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

Example

Change the second value by replacing it with *two* new values:

```
thislist = ["apple", "banana", "cherry"]
thislist[1:2] = ["blackcurrant", "watermelon"]
print(thislist)
```

Note: The length of the list will change when the number of items inserted does not match the number of **items replaced.**

If you insert *less* items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

Example

Change the second and third value by replacing it with *one* value:

```
thislist = ["apple", "banana", "cherry"]
thislist[1:3] = ["watermelon"]
print(thislist)
```

Insert Items

To insert a new list item, without replacing any of the existing values, we can use the insert() method.

The insert() method inserts an item at the specified index:

Example

Insert "watermelon" as the third item:

```
thislist = ["apple", "banana", "cherry"]
thislist.insert(2, "watermelon")
print(thislist)
```

Output

['apple', 'banana', 'watermelon', 'cherry']

Note: As a result of the example above, the list will now contain 4 items.

Append Items

To add an item to the end of the list, use the append() method:

Example

Using the append() method to append an item:

```
thislist = ["apple", "banana", "cherry"]
thislist.append("orange")
print(thislist)
```

Insert Items

To insert a list item at a specified index, use the insert() method.

The insert() method inserts an item at the specified index:

Example

Insert an item as the second position:

```
thislist = ["apple", "banana", "cherry"]
thislist.insert(1, "orange")
print(thislist)
```

Note: As a result of the examples above, the lists will now contain 4 items.

Extend List

To append elements from *another list* to the current list, use the extend() method.

Example

Add the elements of tropical to thislist:

```
thislist = ["apple", "banana", "cherry"]
tropical = ["mango", "pineapple", "papaya"]
```

```
thislist.extend(tropical)
print(thislist)
```

The elements will be added to the *end* of the list.

Add Any Iterable

The extend() method does not have to append *lists*, you can add any iterable object (tuples, sets, dictionaries etc.).

Example

Add elements of a tuple to a list:

```
thislist = ["apple", "banana", "cherry"]
thistuple = ("kiwi", "orange")
thislist.extend(thistuple)
print(thislist)
```

Remove Specified Item

The remove() method removes the specified item.

Example

Remove "banana":

```
thislist = ["apple", "banana", "cherry"]
thislist.remove("banana")
print(thislist)
```

Remove Specified Index

The pop() method removes the specified index.

Example

Remove the second item:

```
thislist = ["apple", "banana", "cherry"]
thislist.pop(1)
print(thislist)
```

Lecture No. 18

If you do not specify the index, the pop() method removes the last item.

Example

Remove the last item:

```
thislist = ["apple", "banana", "cherry"]
thislist.pop()
print(thislist)
```

The del keyword also removes the specified index:

Example

Remove the first item:

```
thislist = ["apple", "banana", "cherry"]
del thislist[0]
print(thislist)
```

The del keyword can also delete the list completely.

Example

Delete the entire list:

```
thislist = ["apple", "banana", "cherry"]
del thislist
```

Clear the List

The clear() method empties the list.

The list still remains, but it has no content.

Example

Clear the list content:

```
thislist = ["apple", "banana", "cherry"]
thislist.clear()
print(thislist)
```

Loop Through a List

You can loop through the list items by using a for loop:

Example

Print all items in the list, one by one:

```
thislist = ["apple", "banana", "cherry"]
for x in thislist:
  print(x)
```

Loop Through the Index Numbers

You can also loop through the list items by referring to their index number.

Use the range() and len() functions to create a suitable iterable.

ExampleGet your own Python Server

Print all items by referring to their index number:

```
thislist = ["apple", "banana", "cherry"]
for i in range(len(thislist)):
  print(thislist[i])
```

The iterable created in the example above is [0, 1, 2].

Using a While Loop

Use the len() function to determine the length of the list, then start at 0 and loop your way through the list items by referring to their indexes.

Remember to increase the index by 1 after each iteration.

Example

Print all items, using a while loop to go through all the index numbers

```
thislist = ["apple", "banana", "cherry"]
i = 0
while i < len(thislist):
  print(thislist[i])
  i = i + 1
```

Looping Using List Comprehension

List Comprehension offers the shortest syntax for looping through lists:

Example

A short hand for loop that will print all items in a list:

```
thislist = ["apple", "banana", "cherry"]
[print(x) for x in thislist]
```

Sort List Alphanumerically

List objects have a sort() method that will sort the list alphanumerically, ascending, by default:

ExampleGet your own Python Server

Sort the list alphabetically:

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort()
print(thislist)
```

ExampleGet your own Python Server

Sort the list numerically:

```
thislist = [100, 50, 65, 82, 23]
thislist.sort()
print(thislist)
```

Sort Descending

To sort descending, use the keyword argument reverse = True:

Example

Sort the list descending:

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort(reverse = True)
print(thislist)
```

Example

Sort the list descending:

```
thislist = [100, 50, 65, 82, 23]
thislist.sort(reverse = True)
print(thislist)
```

Customize Sort Function

You can also customize your own function by using the keyword argument key = *function*.

The function will return a number that will be used to sort the list (the lowest number first):

ExampleGet your own Python Server

Sort the list based on how close the number is to 50:

```
def myfunc(n):
  return abs(n - 50)
thislist = [100, 50, 65, 82, 23]
thislist.sort(key = myfunc)
print(thislist)
```

Case Insensitive Sort

By default the sort() method is case sensitive, resulting in all capital letters being sorted before lower case letters:

Example

Case sensitive sorting can give an unexpected result:

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.sort()
print(thislist)
```

Luckily we can use built-in functions as key functions when sorting a list.

So if you want a case-insensitive sort function, use str.lower as a key function:

Example

Perform a case-insensitive sort of the list:

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.sort(key = str.lower)
print(thislist)
```

Reverse Order

What if you want to reverse the order of a list, regardless of the alphabet?

The reverse() method reverses the current sorting order of the elements.

Example

Reverse the order of the list items:

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.reverse()
print(thislist)
```

Copy a List

You cannot copy a list simply by typing list2 = list1, because: list2 will only be a *reference* to list1, and changes made in list1 will automatically also be made in list2.

Example

Make a copy of a list with the copy() method:

```
thislist = ["apple", "banana", "cherry"]
mylist = thislist.copy()
print(mylist)
```

Another way to make a copy is to use the built-in method list().

Example

Make a copy of a list with the list() method:

```
thislist = ["apple", "banana", "cherry"]
mylist = list(thislist)
print(mylist)
```

Join Two Lists

There are several ways to join, or concatenate, two or more lists in Python.

One of the easiest ways are by using the + operator.

Example

Join two list:

```
list1 = ["a", "b", "c"]
list2 = [1, 2, 3]
list3 = list1 + list2
print(list3)
```

Another way to join two lists is by appending all the items from list2 into list1, one by one:

Example

Append list2 into list1:

```
list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]
for x in list2:
  list1.append(x)
print(list1)
```

Or you can use the extend() method, which purpose is to add elements from one list to another list:

Example

Use the extend() method to add list2 at the end of list1:

```
list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]
list1.extend(list2)
print(list1)
```

List Methods

Python has a set of built-in methods that you can use on lists.

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

Python List append() Method

Example

Add an element to the fruits list:

```
fruits = ['apple', 'banana', 'cherry']
fruits.append("orange")
```

Definition and Usage

The append() method appends an element to the end of the list.

Syntax

list.append(*elmnt*)

Parameter Values

Parameter	Description

elmnt	Required. An element of any type (string, number, object etc.)

Example

Add a list to a list:

```
a = ["apple", "banana", "cherry"]
b = ["Ford", "BMW", "Volvo"]
a.append(b)
```

Python List pop() Method

Remove the second element of the fruit list:

```
fruits = ['apple', 'banana', 'cherry']
fruits.pop(1)
```

Definition and Usage

The pop() method removes the element at the specified position.

Syntax

list.pop(*pos*)

Parameter Values

Parameter	**Description**
pos	Optional. A number specifying the position of the element you want to remove, default value is -1, which returns the last item

Example

Return the removed element:

```
fruits = ['apple', 'banana', 'cherry']

x = fruits.pop(1)
```

Note: The pop() method returns removed value.

Python List sort() Method

Sort the list alphabetically:

```
cars = ['Ford', 'BMW', 'Volvo']
cars.sort()
```

Definition and Usage

The sort() method sorts the list ascending by default.

You can also make a function to decide the sorting criteria(s).

Syntax

list.sort(reverse=True|False, key=myFunc)

Parameter Values

Parameter	Description
reverse	Optional. reverse=True will sort the list descending. Default is reverse=False
key	Optional. A function to specify the sorting criteria(s)

Sort the list descending:

```
cars = ['Ford', 'BMW', 'Volvo']
cars.sort(reverse=True)
```

Example

```
# A function that returns the length of the value:
def myFunc(e):
  return len(e)
cars = ['Ford', 'Mitsubishi', 'BMW', 'VW']
cars.sort(key=myFunc)
```

Example

Sort a list of dictionaries based on the "year" value of the dictionaries:

```
def myFunc(e):
  return e['year']
cars = [
  {'car': 'Ford', 'year': 2005},
  {'car': 'Mitsubishi', 'year': 2000},
  {'car': 'BMW', 'year': 2019},
  {'car': 'VW', 'year': 2011}
]
cars.sort(key=myFunc)
```

Lecture No. 19

Tuples: Create, Indexing ans Slicing, Operation on tuples.

Tuple

Tuples are used to store multiple items in a single variable.
Tuple is one of 4 built-in data types in Python used to store collections of data, the other 3 are List, Set, and Dictionary, all with different qualities and usage.

A tuple is a collection which is ordered and **unchangeable**.

Tuples are written with round brackets.

Example
Create a Tuple:
```
thistuple = ("apple", "banana", "cherry")
print(thistuple)
```

Tuple Items
Tuple items are ordered, unchangeable, and allow duplicate values.
Tuple items are indexed, the first item has index [0], the second item has index [1] etc.

Ordered
When we say that tuples are ordered, it means that the items have a defined order, and that order will not change.

Unchangeable
Tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created.

Allow Duplicates
Since tuples are indexed, they can have items with the same value:
Example
Tuples allow duplicate values:
```
thistuple = ("apple", "banana", "cherry", "apple", "cherry")
print(thistuple)
```

Tuple Length
To determine how many items a tuple has, use the len() function:

Example
Print the number of items in the tuple:
```
thistuple = ("apple", "banana", "cherry")
print(len(thistuple))
```

Create Tuple With One Item
To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.
Example
One item tuple, remember the comma:

```
thistuple = ("apple",)
print(type(thistuple))

#NOT a tuple
thistuple = ("apple")
print(type(thistuple))
```

Tuple Items - Data Types
Tuple items can be of any data type:

Example
String, int and boolean data types:

```
tuple1 = ("apple", "banana", "cherry")
tuple2 = (1, 5, 7, 9, 3)
tuple3 = (True, False, False)
```

A tuple can contain different data types:

Example
A tuple with strings, integers and boolean values:

```
tuple1 = ("abc", 34, True, 40, "male")
```

type()
From Python's perspective, tuples are defined as objects with the data type 'tuple':
<class 'tuple'>

Example
What is the data type of a tuple?

```
mytuple = ("apple", "banana", "cherry")
print(type(mytuple))
```

The tuple() Constructor
It is also possible to use the tuple() constructor to make a tuple.
Example
Using the tuple() method to make a tuple:

```
thistuple = tuple(("apple", "banana", "cherry")) # note the double round-brackets
print(thistuple)
```

Python Collections (Arrays)
There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.

- **Dictionary** is a collection which is ordered** and changeable. No duplicate members.

When choosing a collection type, it is useful to understand the properties of that type. Choosing the right type for a particular data set could mean retention of meaning, and, it could mean an increase in efficiency or security.

Access Tuple Items
You can access tuple items by referring to the index number, inside square brackets:

Example
Print the second item in the tuple:
```
thistuple = ("apple", "banana", "cherry")
print(thistuple[1])
```

Note: The first item has index 0.

Negative Indexing
Negative indexing means start from the end.
-1 refers to the last item, -2 refers to the second last item etc.
Example
Print the last item of the tuple:
```
thistuple = ("apple", "banana", "cherry")
print(thistuple[-1])
```

Range of Indexes
You can specify a range of indexes by specifying where to start and where to end the range.
When specifying a range, the return value will be a new tuple with the specified items.

Example
Return the third, fourth, and fifth item:
```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[2:5])
```
Note: The search will start at index 2 (included) and end at index 5 (not included).
Remember that the first item has index 0.
By leaving out the start value, the range will start at the first item:

Example
This example returns the items from the beginning to, but NOT included, "kiwi":
```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[:4])
```
By leaving out the end value, the range will go on to the end of the list:

Example
This example returns the items from "cherry" and to the end:
```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[2:])
```

Range of Negative Indexes
Specify negative indexes if you want to start the search from the end of the tuple:

Example
This example returns the items from index -4 (included) to index -1 (excluded)
```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[-4:-1])
```

Check if Item Exists
To determine if a specified item is present in a tuple use the in keyword:
Example
Check if "apple" is present in the tuple:
```
thistuple = ("apple", "banana", "cherry")
if "apple" in thistuple:
  print("Yes, 'apple' is in the fruits tuple")
```

Python - Update Tuples

Tuples are unchangeable, meaning that you cannot change, add, or remove items once the tuple is created. But there are some workarounds.

Change Tuple Values
Once a tuple is created, you cannot change its values. Tuples are **unchangeable**, or **immutable** as it also is called.
But there is a workaround. You can convert the tuple into a list, change the list, and convert the list back into a tuple.

Example
Convert the tuple into a list to be able to change it:
```
x = ("apple", "banana", "cherry")
y = list(x)
y[1] = "kiwi"
x = tuple(y)

print(x)
```

Add Items
Since tuples are immutable, they do not have a build-in append() method, but there are other ways to add items to a tuple.

1. **Convert into a list**: Just like the workaround for *changing* a tuple, you can convert it into a list, add your item(s), and convert it back into a tuple.

Example
Convert the tuple into a list, add "orange", and convert it back into a tuple:
```
thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.append("orange")
thistuple = tuple(y)
```

2. **Add tuple to a tuple**. You are allowed to add tuples to tuples, so if you want to add one item, (or many), create a new tuple with the item(s), and add it to the existing tuple:

Example
Create a new tuple with the value "orange", and add that tuple:

```
thistuple = ("apple", "banana", "cherry")
y = ("orange",)
thistuple += y

print(thistuple)
```

Note: When creating a tuple with only one item, remember to include a comma after the item, otherwise it will not be identified as a tuple.

Remove Items

Note: You cannot remove items in a tuple.

Tuples are **unchangeable**, so you cannot remove items from it, but you can use the same workaround as we used for changing and adding tuple items:

Example
Convert the tuple into a list, remove "apple", and convert it back into a tuple:

```
thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.remove("apple")
thistuple = tuple(y)
```

Or you can delete the tuple completely:

Example
The del keyword can delete the tuple completely:

```
thistuple = ("apple", "banana", "cherry")
del thistuple
print(thistuple) #this will raise an error because the tuple no longer exists
```

Unpacking a Tuple
When we create a tuple, we normally assign values to it. This is called "packing" a tuple:

Example
Packing a tuple:

```
fruits = ("apple", "banana", "cherry")
```

But, in Python, we are also allowed to extract the values back into variables. This is called "unpacking":

Example
Unpacking a tuple:

```
fruits = ("apple", "banana", "cherry")

(green, yellow, red) = fruits

print(green)
print(yellow)
print(red)
```

Note: The number of variables must match the number of values in the tuple, if not, you must use an asterisk to collect the remaining values as a list.

Using Asterisk*
If the number of variables is less than the number of values, you can add an * to the variable name and the values will be assigned to the variable as a list:

Example
Assign the rest of the values as a list called "red":

```
fruits = ("apple", "banana", "cherry", "strawberry", "raspberry")

(green, yellow, *red) = fruits

print(green)
print(yellow)
print(red)
```

If the asterisk is added to another variable name than the last, Python will assign values to the variable until the number of values left matches the number of variables left.

Example
Add a list of values the "tropic" variable:

```
fruits = ("apple", "mango", "papaya", "pineapple", "cherry")

(green, *tropic, red) = fruits

print(green)
print(tropic)
print(red)
```

Loop Through a Tuple
You can loop through the tuple items by using a for loop.

Example
Iterate through the items and print the values:

```
thistuple = ("apple", "banana", "cherry")
for x in thistuple:
  print(x)
```

Lecture No. 20

Loop Through the Index Numbers
You can also loop through the tuple items by referring to their index number.
Use the range() and len() functions to create a suitable iterable.

Example
Print all items by referring to their index number:

```
thistuple = ("apple", "banana", "cherry")
for i in range(len(thistuple)):
  print(thistuple[i])
```

Join Two Tuples
To join two or more tuples you can use the + operator:
Example
Join two tuples:

```
tuple1 = ("a", "b" , "c")
tuple2 = (1, 2, 3)

tuple3 = tuple1 + tuple2
print(tuple3)
```

Multiply Tuples
If you want to multiply the content of a tuple a given number of times, you can use the * operator:
Example
Multiply the fruits tuple by 2:

```
fruits = ("apple", "banana", "cherry")
mytuple = fruits * 2
print(mytuple)
```

Tuple Methods

Python has two built-in methods that you can use on tuples.

Method	Description
count()	Returns the number of times a specified value occurs in a tuple
index()	Searches the tuple for a specified value and returns the position of where it was found

Python Tuple count() Method

Example
Return the number of times the value 5 appears in the tuple:

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
```

```
x = thistuple.count(5)

print(x)
```

Definition and Usage
The count() method returns the number of times a specified value appears in the tuple.

Syntax
tuple.count(*value*)
Parameter Values

Parameter	Description
value	Required. The item to search for

Python Tuple index() Method

Example
Search for the first occurrence of the value 8, and return its position:

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)

x = thistuple.index(8)

print(x)
```

Definition and Usage
The index() method finds the first occurrence of the specified value.
The index() method raises an exception if the value is not found.

Syntax
tuple.index(*value*)
Parameter Values

Parameter	Description
value	Required. The item to search for

Methods

Python program to create a tuple without using parentheses

```
# Creating a tuple

tuple_ = 4, 5.7, "Tuples", ["Python", "Tuples"]

# Displaying the tuple created

print(tuple_)
```

Checking the data type of object tuple

```
print(type(tuple_) )
# Trying to modify tuple_
try:
   tuple_[1] = 4.2
except:
   print(TypeError )
```

Output:

```
(4, 5.7, 'Tuples', ['Python', 'Tuples'])
<class 'tuple'>
<class 'TypeError'>
```

The construction of a tuple from a single member might be hard.

Simply adding parenthesis around the element is insufficient. To be recognised as a tuple, the element must be followed by a comma.

Python program to show how to access tuple elements

```
# Creating a tuple
tuple_ = ("Python", "Tuple", "Ordered", "Collection")
print(tuple_[0])
print(tuple_[1])
# trying to access element index more than the length of a tuple
try:
   print(tuple_[5])
except Exception as e:
   print(e)
# trying to access elements through the index of floating data type
try:
   print(tuple_[1.0])
except Exception as e:
   print(e)
# Creating a nested tuple
nested_tuple = ("Tuple", [4, 6, 2, 6], (6, 2, 6, 7))

# Accessing the index of a nested tuple
print(nested_tuple[0][3])
print(nested_tuple[1][1])
```

Output:

```
Python
Tuple
tuple index out of range
tuple indices must be integers or slices, not float
l
6
```

Negative Indexing
Python's sequence objects support negative indexing.
The last item of the collection is represented by -1, the second last item by -2, and so on.

```
# Python program to show how negative indexing works in Python tuples
# Creating a tuple
tuple_ = ("Python", "Tuple", "Ordered", "Collection")

# Printing elements using negative indices
print("Element at -1 index: ", tuple_[-1])
print("Elements between -4 and -1 are: ", tuple_[-4:-1])
```

Output:
Element at -1 index: Collection
Elements between -4 and -1 are: ('Python', 'Tuple', 'Ordered')

Slicing
In Python, tuple slicing is a common practise and the most popular method for programmers to handle practical issues. Think about a Python tuple. To access a variety of elements in a tuple, you must slice it. One approach is to use the colon as a straightforward slicing operator (:).
We can use a slicing operator, a colon (:), to access a range of tuple elements.

```
# Python program to show how slicing works in Python tuples
# Creating a tuple
tuple_ = ("Python", "Tuple", "Ordered", "Immutable", "Collection", "Objects")
# Using slicing to access elements of the tuple
print("Elements between indices 1 and 3: ", tuple_[1:3])

# Using negative indexing in slicing
print("Elements between indices 0 and -4: ", tuple_[:-4])
# Printing the entire tuple by using the default start and end values.
print("Entire tuple: ", tuple_[:])
```

Output:
Elements between indices 1 and 3: ('Tuple', 'Ordered')
Elements between indices 0 and -4: ('Python', 'Tuple')
Entire tuple: ('Python', 'Tuple', 'Ordered', 'Immutable', 'Collection', 'Objects')

Deleting a Tuple
A tuple's components cannot be altered, as was previously said. As a result, we are unable to get rid of or remove tuple components.
However, a tuple can be totally deleted with the keyword del.

```
# Python program to show how to delete elements of a Python tuple
# Creating a tuple
tuple_ = ("Python", "Tuple", "Ordered", "Immutable", "Collection", "Objects")
# Deleting a particular element of the tuple
```

```
try:
   del tuple_[3]
   print(tuple_)
except Exception as e:
   print(e)
# Deleting the variable from the global space of the program
del tuple_
# Trying accessing the tuple after deleting it
try:
   print(tuple_)
except Exception as e:
   print(e)
```

Output:

'tuple' object does not support item deletion
name 'tuple_' is not defined

Repetition Tuples in Python

```
# Python program to show repetition in tuples
tuple_ = ('Python',"Tuples")
print("Original tuple is: ", tuple_)
# Repeting the tuple elements
tuple_ = tuple_ * 3
print("New tuple is: ", tuple_)
```

Output:

Original tuple is: ('Python', 'Tuples')
New tuple is: ('Python', 'Tuples', 'Python', 'Tuples', 'Python', 'Tuples')
Tuple Methods
Python Tuples is a collection of immutable objects that is more like to a list. Python offers a few ways to work with tuples. These two approaches will be thoroughly covered in this essay with the aid of some **examples.**

Count () Method

The number of times the specified element occurs in the tuple is returned by the count () function of Tuple.
Code

```
# Creating tuples
T1 = (0, 1, 5, 6, 7, 2, 2, 4, 2, 3, 2, 3, 1, 3, 2)
T2 = ('python', 'java', 'python', 'Tpoint', 'python', 'java')
# counting the appearance of 3
res = T1.count(2)
print('Count of 2 in T1 is:', res)
# counting the appearance of java
res = T2.count('java')
print('Count of Java in T2 is:', res)
```

Output:

Count of 2 in T1 is: 5
Count of java in T2 is: 2

Index() Method:
The first instance of the requested element from the tuple is returned by the Index() function.
Parameters:

The element to be looked for.
begin (Optional): the index used as the starting point for searching
final (optional): The last index up until which the search is conducted
Index() Method

```
# Creating tuples
Tuple_data = (0, 1, 2, 3, 2, 3, 1, 3, 2)
# getting the index of 3
res = Tuple_data.index(3)
print('First occurrence of 1 is', res)

# getting the index of 3 after 4th
# index
res = Tuple_data.index(3, 4)
print('First occurrence of 1 after 4th index is:', res)
```

Output:
First occurrence of 1 is 2
First occurrence of 1 after 4th index is: 6

Tuple Membership Test
Using the in keyword, we can determine whether an item is present in the given tuple or not.

```
# Python program to show how to perform membership test for tuples
# Creating a tuple
tuple_ = ("Python", "Tuple", "Ordered", "Immutable", "Collection", "Ordered")
# In operator
print('Tuple' in tuple_)
print('Items' in tuple_)
# Not in operator
print('Immutable' not in tuple_)
print('Items' not in tuple_)
```

Output:
True
False
False
True

Iterating Through a Tuple

We can use a for loop to iterate through each element of a tuple.

```
# Python program to show how to iterate over tuple elements
# Creating a tuple
tuple_ = ("Python", "Tuple", "Ordered", "Immutable")
# Iterating over tuple elements using a for loop
for item in tuple_:
    print(item)
```

Output:

```
Python
Tuple
Ordered
Immutable
```

Changing a Tuple

Tuples, as opposed to lists, are immutable objects.

This suggests that we are unable to change a tuple's elements once they have been defined. The nested elements of an element can be changed, though, if the element itself is a changeable data type like a list.

A tuple can be assigned to many values (reassignment).

```
# Python program to show that Python tuples are immutable objects
# Creating a tuple
tuple_ = ("Python", "Tuple", "Ordered", "Immutable", [1,2,3,4])
# Trying to change the element at index 2
try:
    tuple_[2] = "Items"
    print(tuple_)
except Exception as e:
    print( e )
# But inside a tuple, we can change elements of a mutable object
tuple_[-1][2] = 10
print(tuple_)
# Changing the whole tuple
tuple_ = ("Python", "Items")
print(tuple_)
```

Output:

```
'tuple' object does not support item assignment
('Python', 'Tuple', 'Ordered', 'Immutable', [1, 2, 10, 4])
('Python', 'Items')
```

Lecture No. 21

Dictionary: Create,add, and replace values, operation on dictionaries

Dictionary
Dictionaries are used to store data values in key:value pairs.

A dictionary is a collection which is ordered*, changeable and do not allow duplicates.
As of Python version 3.7, dictionaries are *ordered*. In Python 3.6 and earlier, dictionaries are *unordered*.
Dictionaries are written with curly brackets, and have keys and values:

Example
Create and print a dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict)
```

Dictionary Items
Dictionary items are ordered, changeable, and does not allow duplicates.
Dictionary items are presented in key:value pairs, and can be referred to by using the key name.

Example
Print the "brand" value of the dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict["brand"])
```

Ordered or Unordered?
As of Python version 3.7, dictionaries are *ordered*. In Python 3.6 and earlier, dictionaries are *unordered*.
When we say that dictionaries are ordered, it means that the items have a defined order, and that order will not change.
Unordered means that the items does not have a defined order, you cannot refer to an item by using an index.

Changeable
Dictionaries are changeable, meaning that we can change, add or remove items after the dictionary has been created.

Duplicates Not Allowed

Dictionaries cannot have two items with the same key:
Example
Duplicate values will overwrite existing values:
```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964,
  "year": 2020
}
print(thisdict)
```

Dictionary Length
To determine how many items a dictionary has, use the len() function:
Example
Print the number of items in the dictionary:
```
print(len(thisdict))
```

Dictionary Items - Data Types
The values in dictionary items can be of any data type:
Example
String, int, boolean, and list data types:
```
thisdict = {
  "brand": "Ford",
  "electric": False,
  "year": 1964,
  "colors": ["red", "white", "blue"]
}
```

type()
From Python's perspective, dictionaries are defined as objects with the data type 'dict':
<class 'dict'>

Example
Print the data type of a dictionary:
```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(type(thisdict))
```

The dict() Constructor
It is also possible to use the dict() constructor to make a dictionary.

Example
Using the dict() method to make a dictionary:
```
thisdict = dict(name = "John", age = 36, country = "Norway")
print(thisdict)
```

Python Collections (Arrays)
There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.
- **Dictionary** is a collection which is ordered** and changeable. No duplicate members.

When choosing a collection type, it is useful to understand the properties of that type. Choosing the right type for a particular data set could mean retention of meaning, and, it could mean an increase in efficiency or security.

Accessing Items
You can access the items of a dictionary by referring to its key name, inside square brackets:
Example
Get the value of the "model" key:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
x = thisdict["model"]
```

There is also a method called get() that will give you the same result:
Example
Get the value of the "model" key:

```
x = thisdict.get("model")
```

Get Keys
The keys() method will return a list of all the keys in the dictionary.
Example
Get a list of the keys:

```
x = thisdict.keys()
```

The list of the keys is a *view* of the dictionary, meaning that any changes done to the dictionary will be reflected in the keys list.
Example
Add a new item to the original dictionary, and see that the keys list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}

x = car.keys()
```

```
print(x) #before the change

car["color"] = "white"

print(x) #after the change
```

Get Values
The values() method will return a list of all the values in the dictionary.
Example
Get a list of the values:
x = thisdict.values()
The list of the values is a *view* of the dictionary, meaning that any changes done to the dictionary will be reflected in the values list.
Example
Make a change in the original dictionary, and see that the values list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}

x = car.values()

print(x) #before the change

car["year"] = 2020

print(x) #after the change
```

Example
Add a new item to the original dictionary, and see that the values list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}

x = car.values()

print(x) #before the change

car["color"] = "red"

print(x) #after the change
```

Get Items
The items() method will return each item in a dictionary, as tuples in a list.
Example
Get a list of the key:value pairs

x = thisdict.items()
The returned list is a *view* of the items of the dictionary, meaning that any changes done to the dictionary will be reflected in the items list.

Example

Make a change in the original dictionary, and see that the items list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}

x = car.items()

print(x) #before the change

car["year"] = 2020

print(x) #after the change
```

Example

Add a new item to the original dictionary, and see that the items list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}

x = car.items()

print(x) #before the change

car["color"] = "red"

print(x) #after the change
```

Check if Key Exists

To determine if a specified key is present in a dictionary use the in keyword:

Example

Check if "model" is present in the dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
if "model" in thisdict:
  print("Yes, 'model' is one of the keys in the thisdict dictionary")
```

Lecture No. 22

Change Values

You can change the value of a specific item by referring to its key name:

Example

Change the "year" to 2018:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict["year"] = 2018
```

Update Dictionary

The update() method will update the dictionary with the items from the given argument.
The argument must be a dictionary, or an iterable object with key:value pairs.

Example

Update the "year" of the car by using the update() method:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict.update({"year": 2020})
```

Adding Items

Adding an item to the dictionary is done by using a new index key and assigning a value to it:

Example

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict["color"] = "red"
print(thisdict)
```

Update Dictionary

The update() method will update the dictionary with the items from a given argument. If the item does not exist, the item will be added.
The argument must be a dictionary, or an iterable object with key:value pairs.

Example

Add a color item to the dictionary by using the update() method:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
```

```
}
thisdict.update({"color": "red"})
```

Removing Items
There are several methods to remove items from a dictionary:

Example
The pop() method removes the item with the specified key name:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict.pop("model")
print(thisdict)
```

Example
The popitem() method removes the last inserted item (in versions before 3.7, a random item is removed instead):

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict.popitem()
print(thisdict)
```

Example
The del keyword removes the item with the specified key name:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
del thisdict["model"]
print(thisdict)
```

Example
The del keyword can also delete the dictionary completely:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
del thisdict
print(thisdict) #this will cause an error because "thisdict" no longer exists.
```

Example
The clear() method empties the dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
```

```
}
thisdict.clear()
print(thisdict)
```

Loop Through a Dictionary

You can loop through a dictionary by using a for loop.

When looping through a dictionary, the return value are the *keys* of the dictionary, but there are methods to return the *values* as well.

Example

Print all key names in the dictionary, one by one:

```
for x in thisdict:
  print(x)
```

Example

Print all *values* in the dictionary, one by one:

```
for x in thisdict:
  print(thisdict[x])
```

Example

You can also use the values() method to return values of a dictionary:

```
for x in thisdict.values():
  print(x)
```

Example

You can use the keys() method to return the keys of a dictionary:

```
for x in thisdict.keys():
  print(x)
```

Copy a Dictionary

You cannot copy a dictionary simply by typing dict2 = dict1, because: dict2 will only be a *reference* to dict1, and changes made in dict1 will automatically also be made in dict2.

There are ways to make a copy, one way is to use the built-in Dictionary method copy().

ExampleGet your own Python Server

Make a copy of a dictionary with the copy() method:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
mydict = thisdict.copy()
print(mydict)
```

Another way to make a copy is to use the built-in function dict().

Example

Make a copy of a dictionary with the dict() function:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
```

```
  "year": 1964
}
mydict = dict(thisdict)
print(mydict)
```

Nested Dictionaries

A dictionary can contain dictionaries, this is called nested dictionaries.

Example

Create a dictionary that contain three dictionaries:

```
myfamily = {
  "child1" : {
    "name" : "Emil",
    "year" : 2004
  },
  "child2" : {
    "name" : "Tobias",
    "year" : 2007
  },
  "child3" : {
    "name" : "Linus",
    "year" : 2011
  }
}
```

Or, if you want to add three dictionaries into a new dictionary:

Example

Create three dictionaries, then create one dictionary that will contain the other three dictionaries:

```
child1 = {
  "name" : "Emil",
  "year" : 2004
}
child2 = {
  "name" : "Tobias",
  "year" : 2007
}
child3 = {
  "name" : "Linus",
  "year" : 2011
}

myfamily = {
  "child1" : child1,
  "child2" : child2,
  "child3" : child3
}
```

Access Items in Nested Dictionaries
To access items from a nested dictionary, you use the name of the dictionaries, starting with the outer dictionary:
Example
Print the name of child 2:
print(myfamily["child2"]["name"])

Dictionary Methods
Python has a set of built-in methods that you can use on dictionaries.

Method	Description
clear()	Removes all the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and value
get()	Returns the value of the specified key
items()	Returns a list containing a tuple for each key value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key-value pair
setdefault()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary

Python Dictionary pop() Method

Remove "model" from the dictionary:

```
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}

car.pop("model")

print(car)
```

Definition and Usage
The pop() method removes the specified item from the dictionary.
The value of the removed item is the return value of the pop() method, see example below.
Syntax
dictionary.pop(*keyname, defaultvalue*)
Parameter Values

Parameter	Description
keyname	Required. The keyname of the item you want to remove
defaultvalue	Optional. A value to return if the specified key do not exist. If this parameter is not specified, and the no item with the specified key is found, an error is raised

Example
The value of the removed item is the return value of the pop() method:

```
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
x = car.pop("model")
print(x)
```

Python Dictionary update() Method

Example
Insert an item to the dictionary:

```
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
car.update({"color": "White"})

print(car)
```

Definition and Usage
The update() method inserts the specified items to the dictionary.
The specified items can be a dictionary, or an iterable object with key value pairs.

Syntax
dictionary.update(*iterable*)
Parameter Values

Parameter	Description
iterable	A dictionary or an iterable object with key value pairs, that will be inserted to the dictionary

Lecture No. 23

Sets: Create and operations on set.

Set

Sets are used to store multiple items in a single variable.
Set is one of 4 built-in data types in Python used to store collections of data, the other 3 are List, Tuple, and Dictionary, all with different qualities and usage.
A set is a collection which is *unordered*, *unchangeable**, and *unindexed*.

*** Note:** Set *items* are unchangeable, but you can remove items and add new items.
Sets are written with curly brackets.

Example
Create a Set:
thisset = {"apple", "banana", "cherry"}
print(thisset)
Note: Sets are unordered, so you cannot be sure in which order the items will appear.

Set Items
Set items are unordered, unchangeable, and do not allow duplicate values.

Unordered
Unordered means that the items in a set do not have a defined order.
Set items can appear in a different order every time you use them, and cannot be referred to by index or key.

Unchangeable
Set items are unchangeable, meaning that we cannot change the items after the set has been created.
Once a set is created, you cannot change its items, but you can remove items and add new items.

Duplicates Not Allowed
Sets cannot have two items with the same value.
Example
Duplicate values will be ignored:
thisset = {"apple", "banana", "cherry", "apple"}

print(thisset)
Note: The values True and 1 are considered the same value in sets, and are treated as duplicates:
Example
True and 1 is considered the same value:
thisset = {"apple", "banana", "cherry", True, 1, 2}

print(thisset)
Access Items
You cannot access items in a set by referring to an index or a key.

But you can loop through the set items using a for loop, or ask if a specified value is present in a set, by using the in keyword.
Example
Loop through the set, and print the values:

```
thisset = {"apple", "banana", "cherry"}

for x in thisset:
  print(x)
```

Example
Check if "banana" is present in the set:

```
thisset = {"apple", "banana", "cherry"}

print("banana" in thisset)
```

Change Items
Once a set is created, you cannot change its items, but you can add new items.

Add Items
Once a set is created, you cannot change its items, but you can add new items.
To add one item to a set use the add() method.
Example
Add an item to a set, using the add() method:

```
thisset = {"apple", "banana", "cherry"}

thisset.add("orange")

print(thisset)
```

Add Sets
To add items from another set into the current set, use the update() method.
Example
Add elements from tropical into thisset:

```
thisset = {"apple", "banana", "cherry"}
tropical = {"pineapple", "mango", "papaya"}

thisset.update(tropical)

print(thisset)
```

Add Any Iterable
The object in the update() method does not have to be a set, it can be any iterable object (tuples, lists, dictionaries etc.).
Example
Add elements of a list to at set:

```
thisset = {"apple", "banana", "cherry"}
mylist = ["kiwi", "orange"]
```

```
thisset.update(mylist)

print(thisset)
```

Remove Item

To remove an item in a set, use the remove(), or the discard() method.

Example

Remove "banana" by using the remove() method:

```
thisset = {"apple", "banana", "cherry"}

thisset.remove("banana")

print(thisset)
```

Note: If the item to remove does not exist, remove() will raise an error.

Exampler

Remove "banana" by using the discard() method:

```
thisset = {"apple", "banana", "cherry"}

thisset.discard("banana")

print(thisset)
```

Note: If the item to remove does not exist, discard() will **NOT** raise an error.

You can also use the pop() method to remove an item, but this method will remove a random item, so you cannot be sure what item that gets removed.

The return value of the pop() method is the removed item.

Example

Remove a random item by using the pop() method:

```
thisset = {"apple", "banana", "cherry"}

x = thisset.pop()

print(x)

print(thisset)
```

Note: Sets are *unordered*, so when using the pop() method, you do not know which item that gets removed.

Example

The clear() method empties the set:

```
thisset = {"apple", "banana", "cherry"}

thisset.clear()

print(thisset)
```

Example

The del keyword will delete the set completely:

```
thisset = {"apple", "banana", "cherry"}
```

```
del thisset

print(thisset)
```

Loop Items
You can loop through the set items by using a for loop:
Example
Loop through the set, and print the values:

```
thisset = {"apple", "banana", "cherry"}

for x in thisset:
  print(x)
```

Join Two Sets
There are several ways to join two or more sets in Python.
You can use the union() method that returns a new set containing all items from both sets, or the update() method that inserts all the items from one set into another:

Example
The union() method returns a new set with all items from both sets:

```
set1 = {"a", "b" , "c"}
set2 = {1, 2, 3}

set3 = set1.union(set2)
print(set3)
```

Example
The update() method inserts the items in set2 into set1:

```
set1 = {"a", "b" , "c"}
set2 = {1, 2, 3}

set1.update(set2)
print(set1)
```

Note: Both union() and update() will exclude any duplicate items.

Keep ONLY the Duplicates
The intersection_update() method will keep only the items that are present in both sets.
ExampleGet your own Python Server
Keep the items that exist in both set x, and set y:

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}

x.intersection_update(y)

print(x)
```

The intersection() method will return a *new* set, that only contains the items that are present in both sets.

Lecture No. 24

Example
Return a set that contains the items that exist in both set x, and set y:

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}

z = x.intersection(y)

print(z)
```

Keep All, But NOT the Duplicates
The symmetric_difference_update() method will keep only the elements that are NOT present in both sets.
Example
Keep the items that are not present in both sets:

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}

x.symmetric_difference_update(y)

print(x)
```

The symmetric_difference() method will return a new set, that contains only the elements that are NOT present in both sets.

Example
Return a set that contains all items from both sets, except items that are present in both:

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}

z = x.symmetric_difference(y)

print(z)
```

Note: The values True and 1 are considered the same value in sets, and are treated as duplicates:

Example
True and 1 is considered the same value:

```
x = {"apple", "banana", "cherry", True}
y = {"google", 1, "apple", 2}

z = x.symmetric_difference(y)
```

Set Methods
Python has a set of built-in methods that you can use on sets.

Method	Description
add()	Adds an element to the set
clear()	Removes all the elements from the set
copy()	Returns a copy of the set
difference()	Returns a set containing the difference between two or more sets
difference_update()	Removes the items in this set that are also included in another, specified set
discard()	Remove the specified item
intersection()	Returns a set, that is the intersection of two other sets
intersection_update()	Removes the items in this set that are not present in other, specified set(s)
isdisjoint()	Returns whether two sets have a intersection or not
issubset()	Returns whether another set contains this set or not
issuperset()	Returns whether this set contains another set or not
pop()	Removes an element from the set
remove()	Removes the specified element
symmetric_difference()	Returns a set with the symmetric differences of two sets
symmetric_difference_update()	inserts the symmetric differences from this set and another
union()	Return a set containing the union of sets
update()	Update the set with the union of this set and others

Python Set union() Method

Example
Return a set that contains all items from both sets, duplicates are excluded:

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
```

```
z = x.union(y)

print(z)
```

Definition and Usage
The union() method returns a set that contains all items from the original set, and all items from the specified set(s).

You can specify as many sets you want, separated by commas.
It does not have to be a set, it can be any iterable object.
If an item is present in more than one set, the result will contain only one appearance of this item.

Syntax
set.union(*set1, set2...*)
Parameter Values

Parameter	Description
set1	Required. The iterable to unify with
set2	Optional. The other iterable to unify with. You can compare as many iterables as you like. Separate each iterable with a comma

Example
Unify more than 2 sets:

```
x = {"a", "b", "c"}
y = {"f", "d", "a"}
z = {"c", "d", "e"}

result = x.union(y, z)

print(result)
```

Python Set intersection() Method

Example
Return a set that contains the items that exist in both set x, and set y:

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}

z = x.intersection(y)

print(z)
```

The intersection() method returns a set that contains the similarity between two or more sets.
Meaning: The returned set contains only items that exist in both sets, or in all sets if the comparison is done with more than two sets.

Syntax
set.intersection(*set1, set2 ... etc*)
Parameter Values

Parameter	Description
set1	Required. The set to search for equal items in
set2	Optional. The other set to search for equal items in. You can compare as many sets you like. Separate the sets with a comma

Example
Compare 3 sets, and return a set with items that is present in all 3 sets:

```
x = {"a", "b", "c"}
y = {"c", "d", "e"}
z = {"f", "g", "c"}

result = x.intersection(y, z)

print(result)
```

Python set Union() Method Example:

```
A = {2, 4, 5, 6}

B = {4, 6, 7, 8}



print("A U B:", A.union(B))
```

Output:

```
A U B: {2, 4, 5, 6, 7, 8}
```

Example 1: Working with Python set Union() methods

```
set1 = {2, 4, 5, 6}

set2 = {4, 6, 7, 8}

set3 = {7, 8, 9, 10}



# union of two sets

print("set1 U set2 : ", set1.union(set2))



# union of three sets

print("set1 U set2 U set3 :",
set1.union(set2, set3))
```

Output

```
set1 U set2 :  {2, 4, 5, 6, 7, 8}

set1 U set2 U set3 : {2, 4, 5, 6, 7, 8, 9, 10}
```

Output:

```
set1 U set2 :  {2, 4, 5, 6, 7, 8}

set1 U set2 U set3 : {2, 4, 5, 6, 7, 8, 9, 10}
```

Example 2: Python Set Union Using the | Operator

We can use "|" operator to find the union of the sets.

```
set1 = {2, 4, 5, 6}

set2 = {4, 6, 7, 8}

set3 = {7, 8, 9, 10}



# union of two sets

print("set1 U set2 : ", set1 | set2)



# union of three sets

print("set1 U set2 U set3 :", set1 |set2 |
set3)
```

Output

```
set1 U set2 :  {2, 4, 5, 6, 7, 8}

set1 U set2 U set3 : {2, 4, 5, 6, 7, 8, 9, 10}
```

Output:

```
set1 U set2 :  {2, 4, 5, 6, 7, 8}

set1 U set2 U set3 : {2, 4, 5, 6, 7, 8, 9, 10}
```

Example 3 : Python Set Union() Method on String

```
A = {'ab', 'ba', 'cd', 'dz'}

B = {'cd', 'ab', 'dd', 'za'}



print("A U B:", A.union(B))
```

Output

```
A U B: {'za', 'ab', 'dd', 'dz', 'ba', 'cd'}
```

Example 4 : Python Set Union() Method on multiple set (With 3 Set).

- Python3

```
A = {2, 4, 5, 6}

B = {4, 6, 7, 8}

c = {7, 8, 9, 0}



print("A U B:", A.union(B).union(c))
```